

Spring Framework

What is Spring Framework?

- The **Spring Framework** is a comprehensive, modular platform for developing Java applications. Introduced in 2003, it provides core features like:
 - **Dependency Injection (DI)** via the Inversion of Control (IoC) container
 - **Aspect-Oriented Programming (AOP)**
 - **Transaction management**
 - **MVC architecture** for web apps
 - Integration with ORM tools like Hibernate and JPA
- It's a **toolbox** that gives you full control over your project structure, but it requires manual configuration and setup — especially for web servers, security, and dependency management.

What is Spring Boot?

➤ Spring Boot is a layer on top of the Spring Framework that simplifies the development process. It offers:

- **Auto-configuration** based on your project's dependencies
- **Embedded servers** (Tomcat, Jetty, Netty) — no need to deploy WAR files
- **Opinionated defaults** for security, databases, logging, etc.
- **Production-ready features** like health checks and metrics
- Support for **GraalVM native images** (from Spring Boot 3 onward)

➤ In other words, Spring Boot uses the Spring Framework **under the hood**, but makes it far easier and faster to build real-world applications.

Key differences between Spring and Spring Boot

Feature	Spring Framework	Spring Boot
Setup Complexity	Manual configuration	Auto-configured, ready to run
Web Server	Requires external server (Tomcat, etc.)	Embedded server (Tomcat, Jetty, Netty)
Dependency Management	Must define all dependencies manually	Starter dependencies with curated sets
Development Speed	Slower, verbose	Faster, cleaner, beginner-friendly
Microservice-Readiness	Needs more setup	Ideal for microservices
Project Size Fit	Great for large, complex architectures	Great for small-to-medium and SaaS apps
Learning Curve	Steeper	Easier for beginners

When should you use Spring Framework?

- Spring Framework is ideal when:
 - You need **fine-grained control** over app configuration
 - You're working on a **legacy system** or migrating from Java EE
 - You're building a **monolithic enterprise application** with layered architecture
 - You're using advanced **modularization** or want to design your own app bootstrapping
- However, in most modern use cases — especially **cloud-native apps, SaaS platforms, APIs, and microservices** — using **Spring Boot** is the preferred approach.

When should you use Spring Boot

- Spring Boot is better when:
 - You want to build and deploy apps **fast**
 - You need **REST APIs** or **microservices**
 - You prefer **convention over configuration**
 - You want to ship apps as **standalone JARs** with embedded servers
 - You need built-in support for **Docker**, **Kubernetes**, or **cloud platforms**

Real- world use case Examples

Application Type	Recommended Approach
REST API backend for mobile app	Spring Boot
Legacy ERP system integration	Spring Framework
Microservice in Kubernetes	Spring Boot
Complex monolithic enterprise app	Spring Framework
Rapid MVP or SaaS app	Spring Boot
Custom modular business platform	Spring Framework

Final Thoughts

- Spring Boot and Spring Framework aren't competitors — they're part of the same ecosystem.
 - Use **Spring Boot** when you want to move fast, deploy clean APIs, and scale on the cloud.
 - Use **Spring Framework** when you need full control over configuration, or are working on large, mature codebases.